EX13 - Trajectories with Lists and Tuples - Storing History

Beginner Step-by-Step Guide

This guide follows the real file `ex13_trajectory_lists_tuples.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Modify the simulation so every trajectory state is kept, not only the current state and final result.

Problem variables / elements

Name	Meaning
state	(h,v,t) at one instant
trajectory	list of states

Equation or logic on paper

```
state_k = (h_k, v_k, t_k)
trajectory = [state_0, state_1, ...]
```

Manual check

After the first Euler step: state_0=(0,50,0) and state_1=(5.0,49.019,0.1). trajectory should contain both.

2. What we already know and what is new

Already seen:

- functions
- while
- Euler
- multiple returns

New in this exercise:

- list
- []
- append()
- tuple
- indices [i] and [-1]
- generator expression inside max()
- snapshots

3. New Python concepts

1. List

Syntax: `trajectory = []`

A list is ordered and mutable. It can start empty and grow during the simulation.

2. append

Syntax: `trajectory.append((h, v, t))`

append adds one element at the end. Here the element is a tuple.

3. Tuple

Syntax: `(h, v, t)`

A tuple is used as a compact state snapshot.

4. Negative index

Syntax: `trajectory[-1]`

-1 selects the last element in a sequence.

5. Generator expression

Syntax: `max(state[0] for state in trajectory)`

Generates state[0] from each state without manually creating another list.

4. Code step by step

Step 1. Create empty lists

```
h = []
v = []
t = []
trajectory = []
```

The collections start empty and are filled with append().

Step 2. Store initial state

```
h.append(h0)
v.append(v0)
t.append(t0)
trajectory.append((h0, v0, t0))
```

The initial state is stored so it becomes part of the history.

Step 3. Access current state

```
while h[i] >= 0:
    h_new, v_new, t_new = euler_step(h[i], v[i], t[i], g, dt)
```

Because h,v,t are lists, i tells us which sample is the current state.

Step 4. Append a new snapshot

```
trajectory.append((h_new, v_new, t_new))
```

Each iteration creates a new tuple, preserving earlier states.

Step 5. Extract results

```
max_h_trajectory = max(state[0] for state in trajectory)
v_f_trajectory = trajectory[-1][1]
t_f_trajectory = trajectory[-1][2] - t0
```

state[0] is height, [1] velocity and [2] time. trajectory[-1] is the final state.

5. Execution flow

```
initialize lists
 -> append state_0
 -> Euler loop
 -> append each state
 -> analyze trajectory
```

6. Expected result / check

Values extracted from trajectory should match direct results: max_h~129.92 m, t_f~10.30 s and v_f~-51.04 m/s.

7. Common beginner mistakes

- Confusing append(x) with multiple arguments.
- Storing the same mutable object repeatedly as state.
- Confusing trajectory[-1][1] with trajectory[1][-1].
- Forgetting indices start at 0.

8. If you remember only five things

- [] creates an empty list.
- append adds one item.
- A tuple (h,v,t) represents one state.
- -1 means last.
- Storing history enables post-processing.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.